Hands-On Lab 1.1: Introduction to Notebooks in Databricks

Cells, Languages, dbutils, and How to Read an Error

Module	1 — Introduction to Data Engineering and Databricks
Estimated duration	90 minutes
Format	Individual, guided walkthrough with checkpoints

Overview

Lab 1 got you signed in, attached to compute, and running a short smoke test. This lab slows down and treats the notebook itself as the subject — every mechanic you'll rely on for the rest of the course: creating and organising notebooks, mixing languages in the same file, running Python and SQL and shell commands, setting configuration values correctly, using dbutils, and — just as importantly — reading an error message and recovering from it.

Pre-requisite

- Completion of Hands-On Lab 1 — signed in, attached to compute, comfortable with the sidebar.
- No prior notebook, Python, or SQL experience is assumed beyond what Lab 1's smoke test already touched.

Item	Value	Notes
Catalog	actuarial	Fixed — shared by the whole class
Your schema	actuarial.<your_username>	From Lab 1

Learning Objectives

By the end of this lab, you will be able to:

- Create and organise notebooks, and mix languages within a single notebook using magic commands.
- Run Python code, understand what state persists between cells, and use display() effectively.
- Run SQL both as native cells and from Python via spark.sql().
- Run shell commands with %sh and explain what environment they do and don't share with the rest of the notebook.
- Set configuration values correctly using the right mechanism for the job — Python variables, environment variables, or cluster-level settings — and explain why each one does or doesn't persist where you might expect.
- Use core dbutils functions: fs, widgets, notebook chaining, and secrets.
- Read a real error message, identify its cause, and recover from stale notebook state.

Step-by-Step Guide

Part 1 — Create and Organise a Notebook

- Create a folder under Workspace for this lab's work, e.g. a folder named after yourself if you don't have one already.
- Click + New → Notebook. Name it 01_notebook_basics_<your_username>, set the default language to Python, and attach it to your compute.
- Note the alternative path too: right-click any folder in Workspace and choose Create → Notebook — useful once you're organising many notebooks rather than starting from the sidebar every time.

Note: A notebook's default language is just that — a default. Every cell can override it with a magic command, which is exactly what the rest of this lab does.

Checkpoint: A new notebook exists, is attached to running compute, and lives inside a folder you can find again.

Part 2 — Cells, Magic Commands, and Markdown

Add a markdown cell at the very top of your notebook to document what it's for — a habit worth starting now, before Git makes commit history do some of this job for you:

```
%md
# Lab 1.1 - Notebook Basics
Working notebook for practicing cells, languages, dbutils, and error recovery.
```

Run it with Shift+Enter — this runs the current cell and moves to the next one, creating a new cell if you're at the bottom. Ctrl+Enter runs the current cell in place without moving. Run All, in the toolbar, runs every cell top to bottom in order.

Magic command	Runs this cell as
%python (or nothing, if default language is Python)	Python
%sql	SQL
%sh	A shell command on the driver node

Magic command	Runs this cell as
%md	Rendered Markdown text, not executed code
%run	Inlines another notebook's cells into this one — covered in Part 7

Checkpoint: Your markdown cell renders as formatted text, not as plain code, once run.

Part 3 — Running Python Code

Variables persist across cells for as long as your notebook stays attached to the same compute — this is what makes a notebook feel like one continuous session rather than isolated scripts. Try it:

```
mock_premiums = [1200.50, 980.25, 2350.00, 675.10]
loading_factor = 1.15  # expense loading applied on top of pure premium
```

In a new cell — notice mock_premiums is still available, even though it was defined above:

```
loaded_premiums = [round(p * loading_factor, 2) for p in mock_premiums]
print("Original:", mock_premiums)
print("With loading applied:", loaded_premiums)
```

Now work with real (if small) data using Spark rather than plain Python. The built-in samples catalog is available in every workspace, so this works before you've ingested anything of your own:

```
trips = spark.table("samples.nyctaxi.trips")
display(trips.limit(5))
```

display() renders an interactive, sortable table — and can render charts directly from the same output — while print() only ever produces plain text. Compare directly:

```
from pyspark.sql.functions import col, avg

fare_by_distance = (
    trips
```

```
    .withColumn("distance_bucket", (col("trip_distance") / 2).cast("int") * 2)
    .groupBy("distance_bucket")
    .agg(avg("fare_amount").alias("avg_fare"))
    .orderBy("distance_bucket")
)
display(fare_by_distance)
print(fare_by_distance.collect())
```

Checkpoint: display() shows a formatted, interactive table; print() on the same underlying data shows a flat list of Row objects — you can see why display() is almost always the better choice for exploring data.

Part 4 — Running SQL Code

Even in a Python-default notebook, %sql runs a cell as SQL directly:

```
%sql
SELECT
  pickup_zip,
  ROUND(AVG(fare_amount), 2) AS avg_fare,
  COUNT(*) AS trip_count
FROM samples.nyctaxi.trips
GROUP BY pickup_zip
ORDER BY trip_count DESC
LIMIT 10;
```

The exact same query works from a Python cell via spark.sql(), returning a DataFrame you can keep working with in Python:

```
result = spark.sql("""
    SELECT pickup_zip, ROUND(AVG(fare_amount), 2) AS avg_fare, COUNT(*) AS trip_count
    FROM samples.nyctaxi.trips
    GROUP BY pickup_zip
    ORDER BY trip_count DESC
    LIMIT 10
""")
display(result)
```

SQL cells can also build small, throwaway tables inline with VALUES — genuinely useful for prototyping a query's shape before real data exists to back it:

```
%sql
WITH mock_policies AS (
  SELECT * FROM VALUES
    ('POL-A', 'Home',  250000),
    ('POL-B', 'Home',  480000),
    ('POL-C', 'Motor',  32000),
    ('POL-D', 'Motor',  41500)
  AS t(policy_id, product_type, sum_insured)
)
SELECT
  product_type,
  COUNT(*) AS policy_count,
  ROUND(AVG(sum_insured), 2) AS avg_sum_insured
FROM mock_policies
GROUP BY product_type;
```

Checkpoint: The %sql version and the spark.sql() version of the trips query return identical results, and you've run a query against data that exists only inside the query itself.

Part 5 — Running Shell Commands

%sh runs a command directly on the driver node's operating system — not distributed across your cluster, and not the same environment your Python or SQL cells run in:

```
%sh
echo "Driver node check"
python3 --version
df -h /
```

Important: %sh is genuinely useful for quick diagnostics — disk space, OS-level tool versions, checking a file exists — but don't reach for it to install Python packages. Use %pip install <package>

instead, which installs into the notebook's actual Python environment; a plain %sh pip install often installs into the wrong Python entirely and won't be visible to your Python cells.

Checkpoint: You've run at least one %sh command and can state, in one sentence, why it doesn't have access to the same variables your Python cells do.

Part 6 — Defining Environment Variables (and a Real Gotcha)

There are several different things people mean by "environment variable" in a notebook, and mixing them up is a common source of confusion. Work through all three so you can tell them apart.

First, a Python-level variable set through os.environ — this persists for as long as your Python session stays attached, exactly like any other variable:

```
import os
os.environ["LOADING_FACTOR"] = "1.15"
```

In a later cell, it's still there:

```
print(os.environ["LOADING_FACTOR"])
```

Now check whether a %sh cell can see it:

```
%sh
echo $LOADING_FACTOR
```

What to notice: This prints nothing. os.environ lives inside your Python process; %sh spawns a completely separate shell process that never inherited it. Setting a value in Python does not automatically make it available to shell commands.

Now the reverse gotcha — set a value inside one %sh cell, and check it from another:

```
%sh
export MY_VAR="hello from shell"
echo $MY_VAR
```

That prints correctly. Now, in a separate %sh cell:

```
%sh
echo $MY_VAR
```

What to notice: This also prints nothing. Each %sh cell spawns a brand new shell process — nothing set with export in one %sh cell survives into the next one, even though they're right next to each other in the same notebook.

For configuration that genuinely needs to be shared across everyone attached to a cluster — not just one notebook — classic clusters support environment variables set at the compute level, under the cluster's Advanced Options. These persist for anyone attached to that cluster, but require CAN MANAGE permission on it, and don't apply the same way to serverless compute.

Important: None of these three mechanisms are appropriate for secrets — passwords, API keys, tokens. Never hardcode a sensitive value into a notebook cell, an environment variable, or cluster configuration in plain text. Part 7 covers the correct mechanism.

Checkpoint: You've demonstrated, hands-on, that a Python os.environ value doesn't reach %sh, and that one %sh cell's export doesn't reach another — and can explain why in terms of process boundaries, not just “it didn't work.”

Part 7 — Databricks Utilities (dbutils)

dbutils is a Databricks-specific toolkit available automatically in every notebook — no import required. Start by asking it what it can do:

```
dbutils.help()
```

File system access:

```
dbutils.fs.ls("/")
```

This lists the DBFS root — a safe, always-available check that dbutils.fs works, before Lab 3 introduces Volumes as the primary place you'll actually manage files in Unity Catalog going forward.

Widgets — turning a notebook into something parameterised rather than hardcoded:

```
dbutils.widgets.text("loading_factor_input", "1.15")
```

A text box now appears at the top of your notebook. Read its current value back into Python:

```
loading_factor = float(dbutils.widgets.get("loading_factor_input"))
loaded_premiums = [round(p * loading_factor, 2) for p in mock_premiums]
print(loaded_premiums)
```

Change the widget's value at the top of the notebook and re-run just that last cell — the output changes without editing a single line of code.

Secrets — the correct way to handle sensitive values (reference only; this course hasn't set up a secret scope yet):

```
# dbutils.secrets.get(scope="<scope_name>", key="<key_name>")
# Returns the secret's value, but Databricks automatically redacts it
# from notebook output wherever it's printed or displayed.
```

Notebook chaining — %run vs. dbutils.notebook.run:

Create a second, tiny notebook in the same folder named 01b_helper_<your_username>, containing just this:

```
helper_value = "set inside the helper notebook"
dbutils.notebook.exit("helper finished successfully")
```

Back in your main notebook, try both ways of calling it:

```
%run ./01b_helper_<your_username>
```

```
print(helper_value)  # visible - %run inlines the helper's cells into this notebook's own session
```

```
result = dbutils.notebook.run("./01b_helper_<your_username>", timeout_seconds=60)
```

```
print(result)  # the exit value, e.g. "helper finished successfully"
```

What to notice: %run shares your notebook's variable scope directly — helper_value became yours to use. dbutils.notebook.run instead executes the helper as an isolated job-like run and hands you back only whatever it explicitly passed to dbutils.notebook.exit(); helper_value from that run is not visible to you. Use %run for shared utility code within one logical notebook, and dbutils.notebook.run when you deliberately want isolation — which is exactly the model Databricks Workflows itself uses to orchestrate notebooks later in this course.

Checkpoint: You can state which of the two chaining methods exposes the helper notebook's variables directly, and which one doesn't.

Part 8 — Reading and Recovering From Errors

Trigger a real error on purpose — referencing a table that doesn't exist:

```
%sql
SELECT * FROM actuarial.<your_username>.this_table_does_not_exist;
```

Read the error rather than skimming past it. You should see an error class near the top — something like TABLE_OR_VIEW_NOT_FOUND — followed by details identifying exactly which name it couldn't resolve. That first line is almost always enough to tell you what to fix, before reading any further down the stack trace.

Now trigger a subtler, more common problem — stale notebook state from running cells out of order. Create three small cells:

```
# Cell A
status_threshold = 1000
```

```
# Cell B
print(f"Anything above {status_threshold} is high value")
```

Run Cell A, then Cell B — both work fine. Now go back and edit Cell A to a different number, but don't re-run it, and instead just re-run Cell B by itself.

What to notice: Cell B still prints the old threshold value. Editing a cell's text does nothing until that cell is actually re-run — the notebook's live state only changes when a cell executes, not when you type. This is one of the most common sources of "but I changed that already" confusion.

The reliable fix, any time you suspect stale state: use Run All, or Clear State (from the notebook's menu) followed by Run All, which detaches and reattaches your Python session, wiping every variable and forcing every cell to execute fresh, top to bottom.

Checkpoint: You've seen a real error message and identified its cause from the first line alone, and you've reproduced — and then fixed — a stale-state bug using Run All.

Part 9 — Version History and Good Habits

Databricks automatically saves revisions of a notebook as you work — open the version history from the notebook's menu to see it. This is a safety net worth knowing about now, well before Git is introduced later in the course as the real, deliberate way to track and share changes.

- Document notebooks with markdown cells as you go, not retroactively.
- Before handing a notebook to someone else — or moving on to the next lab — run Clear State and Run All at least once, to confirm it actually works top to bottom and isn't secretly depending on stale state only you have.
- Delete scratch/debugging cells once you're done with them, rather than leaving commented-out attempts scattered through the notebook.

Troubleshooting

Symptom	Likely cause	Fix
A cell shows no Run button or won't execute at all	Notebook isn't attached to any compute, or the attached compute isn't running	Check the compute selector at the top of the notebook; attach or start compute as needed
%sh commands work but %python cells suddenly fail with a connection error	Compute was detached or restarted partway through your session	Reattach, then use Clear State and Run All rather than resuming from wherever you left off

Symptom	Likely cause	Fix
A widget doesn't appear after dbutils.widgets.text(...)	The cell that created it hasn't actually been run yet	Run that specific cell; widgets only appear once their creation code executes
dbutils.notebook.run fails with a path error	The helper notebook path is relative to the wrong location, or the filename doesn't match exactly	Use the same folder for both notebooks during this lab, and confirm the exact filename including your username
print(os.environ[...]) fails with a KeyError	The cell that set it was never run, or Clear State reset the session since then	Re-run the cell that sets os.environ["LOADING_FACTOR"] before the cell that reads it

Outcome

By completing this lab, you have:

- Created and organised a notebook, and mixed Python, SQL, shell, and markdown cells within it using magic commands.
- Run Python and SQL code two different ways each, and used display() to explore data properly.
- Demonstrated, hands-on, exactly which configuration mechanism persists where — Python variables within a session, %sh exports nowhere beyond their own cell, and cluster-level settings across an entire compute resource.
- Used dbutils for file listing, widgets, and notebook chaining, and can explain the difference between %run and dbutils.notebook.run.
- Read a real error message and identified its cause from the first line, and diagnosed and fixed a stale-state bug using Run All.